ON-PREM AI ROUTING · WORKING PROTOTYPE

Your IP never leaves the building. Neither does your budget.

AI-Paladin decides where every prompt runs **before a socket opens**. Protected IP executes on hardware you own, where inference costs **nothing per token**. Everything else is your call, with full sight of exactly which bytes would leave.

0
bytes of protected IP transmitted, in any mode

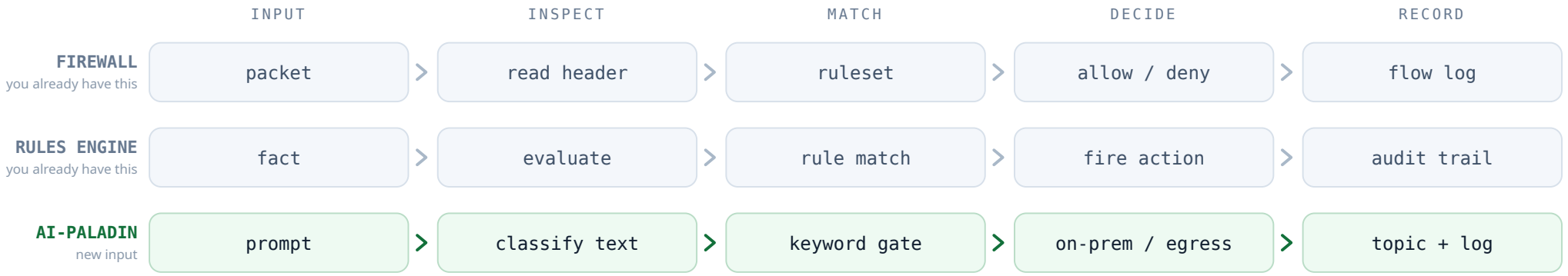
37
protected terms enforced as data, not prompt instructions

73%
of tokens handled on-prem — metered by nobody, billed by nobody

THE CONTROL ALREADY EXISTS

You already own this machine. Just not for prompts.

A prompt is a suggestion. A hook is a wall. You already gate packets with a firewall and business logic with a rules engine — deterministic checks that run before the thing they guard, and that no amount of persuasion talks past. AI-Paladin is that same machine, pointed at the one input your perimeter does not currently inspect.



Same five stages. The only novel part is the input — and it is the one input your perimeter does not currently inspect.

THE PROBLEM

Every prompt is an exfiltration decision. Today it is made by the wrong party.

When an engineer pastes code into a hosted model, the routing decision has already happened — silently, by a human under deadline, with no record. The controls that exist are the weakest kind available:

- **Policy documents**
  A PDF nobody reads at 11pm. Unenforceable, unauditable, and invisible at the moment of the decision.
- **System prompts**
  "Please don't share proprietary code." A suggestion. A manipulated or merely confused agent ignores it and nothing stops the request.
- **Blanket bans**
  Ban the tools and you lose the productivity. Engineers route around it with personal accounts, and now you have no telemetry at all.
- **Trust the vendor**
  A contractual promise about data handling is not a technical control. It cannot be demonstrated to a regulator or an acquirer.

THE INSIGHT

A prompt is a suggestion.
A hook is a wall.

A manipulated or misbehaving agent can ignore *"please don't send that."* It cannot ignore a router that never opens the socket.

Every gate in AI-Paladin is deterministic and lives in version-controlled data — a YAML keyword file and a policy file — evaluated by code the model cannot reach, argue with, or talk its way past. The decision is made on the prompt, by the router, before any model is invoked.

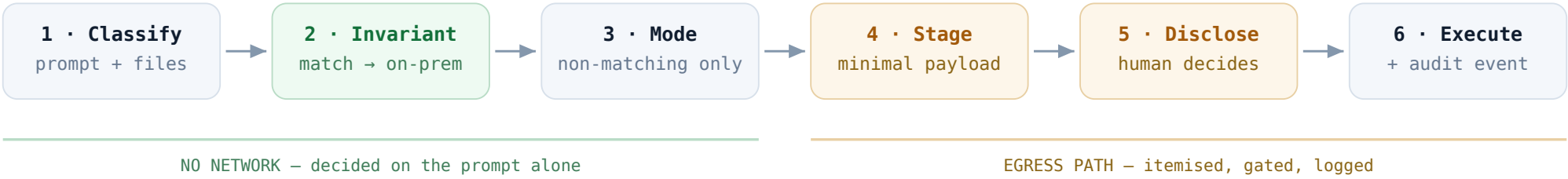
🔒 THE INVARIANT

keyword match ⇒ on-prem.
Always. Not overridable.

This is the entire product thesis in one line. The routing modes below govern only what happens to traffic that does **not** match.

HOW IT WORKS

Six steps. The first three touch no network.



Steps 1–3 are pure computation over the prompt and the files it would carry. A protected task terminates at step 2 and never reaches a socket. Only work that clears the gate is ever staged, and staging is where the payload gets cut to the minimum.

THE CONTROL SURFACE

Three postures. One thing they cannot change.

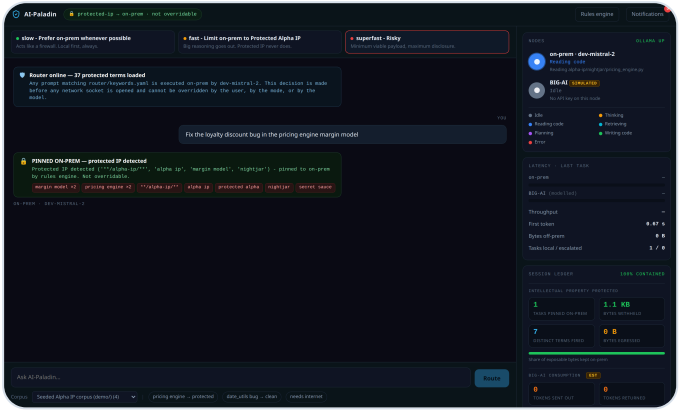
MODE	PROTECTED IP	EVERYTHING ELSE
slow Prefer on-prem	🔒 **on-prem · locked**	Stays on-prem too. Escalation is offered, never automatic. Acts like a firewall — the safest and slowest posture.
fast Limit on-prem to Alpha IP	🔒 **on-prem · locked**	**→ BIG-AI** — full context, human confirms. Large reasoning gets frontier capability; protected IP still cannot follow it out.
superfast Risky	🔒 **on-prem · locked**	**→ BIG-AI** — **minimal slice only**, with a full disclosure window itemising every byte before it moves.

Read down the middle column. The most permissive posture we ship treats protected IP exactly as the most restrictive one does. That is not a default a user can drift away from — it is the one thing the control surface does not expose.

PROOF · 01

The riskiest mode still refuses.

Mode is set to `superfast — Risky`, the most permissive posture available. The prompt names the pricing engine. The router pins it on-prem anyway and names every term that fired.



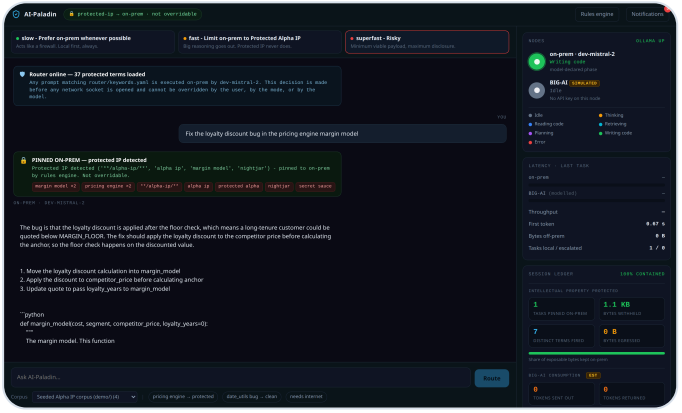
The user cannot select their way around this. The chips are the evidence trail — matched terms, and the file paths they were found in.

PROOF · 02

An agent you can actually watch.

Seven states, derived from the real execution stream — not a progress spinner. File reads carry real paths and byte counts; the working phases are parsed from the model's own output as it produces them.

- **Idle** — Dormant, awaiting work
- **Thinking** — Reasoning about the request
- **Reading code** — Ingesting local files
- **Retrieving** — Reaching outside for information
- **Planning** — Checklists & plans, like a manager
- **Writing code** — Producing code, like a developer
- **Error** — Failed — a human is needed

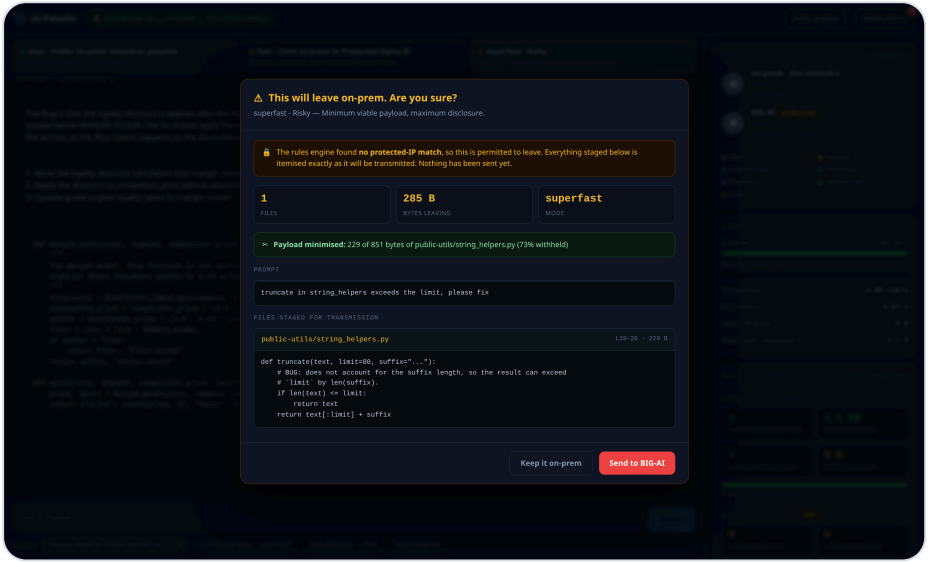


Observed sequence, this run: Reading code → Thinking → Planning → Writing code → Idle. Red stops, green produces, amber deliberates.

PROOF · 03

Nothing leaves unseen.

When work *is* cleared to leave, the human sees the payload before it moves — file, line range, byte count, and the literal text. In `superfast` the router first cuts the payload to the smallest coherent slice: the one function that matters.



229 of 851 bytes — 73% of the file withheld. The default button is "Keep it on-prem".

PROOF · 04

"No" is free, and it is provable.

Declining at the gate cancels the task before transmission. The counter stays at zero and the refusal is written to the audit topic `egress.denied` with its task ID.

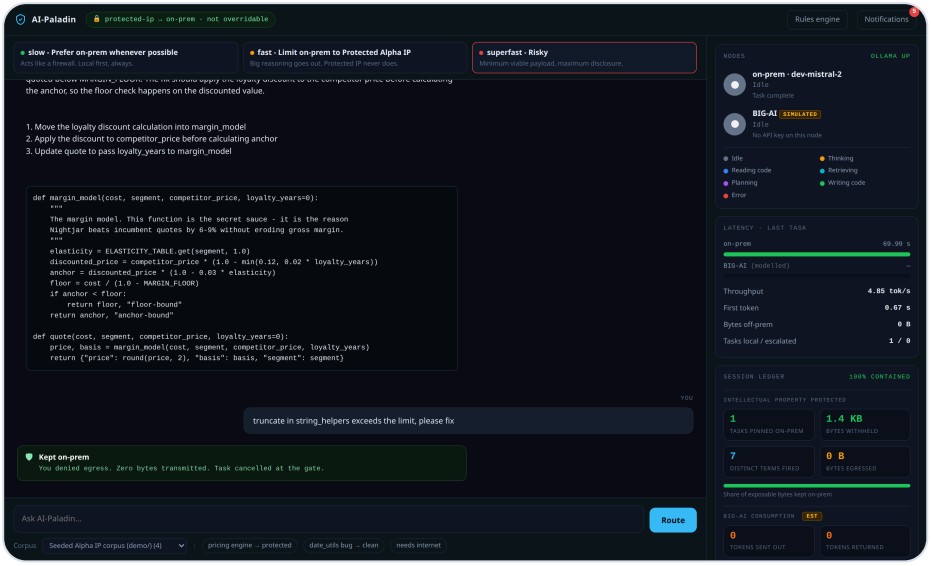
That record is the artifact a regulator, an auditor, or an acquirer's technical diligence team actually wants: not a policy asserting what should have happened, but a log of what did.

0 B

transmitted after a denial

100%

of routing decisions written to an append-only log



Zero bytes transmitted. Task cancelled at the gate.

PROOF · 05

Everything you just saw ran with no GPU at all.

This matters more than any other number in the deck. The on-prem tier is currently doing every bit of that work on **CPU alone** — the integrated graphics sat idle throughout. That is the floor, not the ceiling.

Compute

AMD Ryzen AI 9 HX 370

12 cores / 24 threads

91 GB system RAM

GPU acceleration: none

Radeon 890M iGPU idle, 2 GB

Measured

4.94 tok/s generation

768 tok/s prompt intake

75.0 GB/s effective

~61 s per 300-token answer

median of 3 runs, ±0.01

Model

dev-mistral-2

24B parameters, Q4_K_M

15.18 GB resident

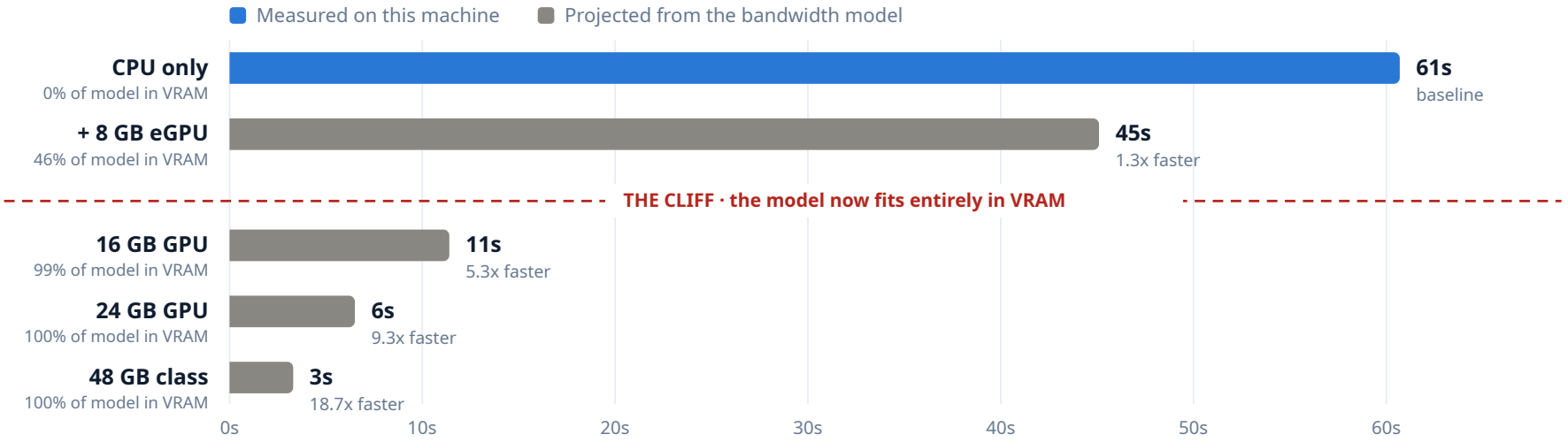
100% in system RAM

60–90 seconds per answer, on commodity CPU, with the accelerator switched off. The next slide is what happens when it is not — and the answer is not the one most people expect.

THE ECONOMICS OF SPEED

It is not a curve. It is a cliff.

Generating a token means reading *every* weight once, so decode speed is set by **memory bandwidth**, not core count: `tokens/sec ≈ bandwidth ÷ model size`.



Seconds to produce a 300-token answer. Measured anchor: 4.94 tok/s on a 15.18 GB model = 75.0 GB/s effective bandwidth.
Projections assume each class delivers ~70% of its peak bandwidth; see bench/project.py.

Why the cliff, and not a ramp

If the model does not fit in VRAM, layers split between card and system RAM — and the system-RAM half never gets faster. That caps an 8 GB card at **1.8× even with an infinitely fast GPU**; realistically it returns **1.3×**.

What that buys, concretely

Crossing the residency threshold takes the answer from **61 s to 11 s**. One tier further reaches **6.5 s — conversational range**, at which point the guarantee costs the user nothing in patience. A four-figure line item, not a research programme.

Projections are a published model, not a claim — assumptions and arithmetic in `bench/project.py`.

PROOF · 06

Two numbers, tracked from the first prompt.

Every organisation adopting AI asks the same two questions within a quarter: **what are we spending outside**, and **what did we manage to keep inside**. Both are counted from the first prompt — the interesting figure is cumulative, and you cannot backfill it.

Protected IP

Keeping **87.7%** of **exposable bytes** on-prem

The other 12.3% is **not leaked IP**: it carried no protected match, and a human read it and approved it at the gate. **Zero bytes of protected IP were transmitted.**

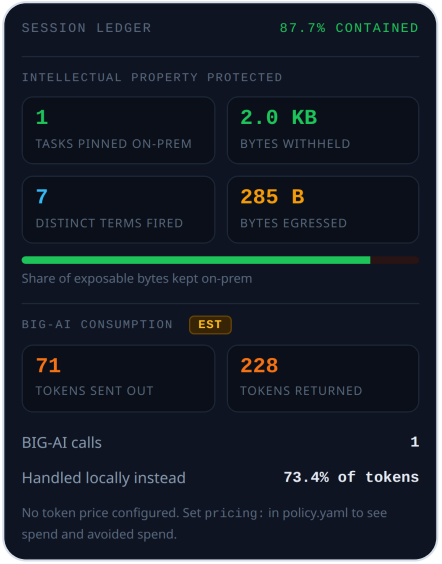
2,033 B withheld — by the gate, by minimisation, and by human refusal

BIG-AI consumption

299 tokens — sent out and returned, counted separately

Hosted providers bill on what you *send* as well as what comes back, so the ledger tracks both directions.

73.4% of all tokens **handled locally** instead



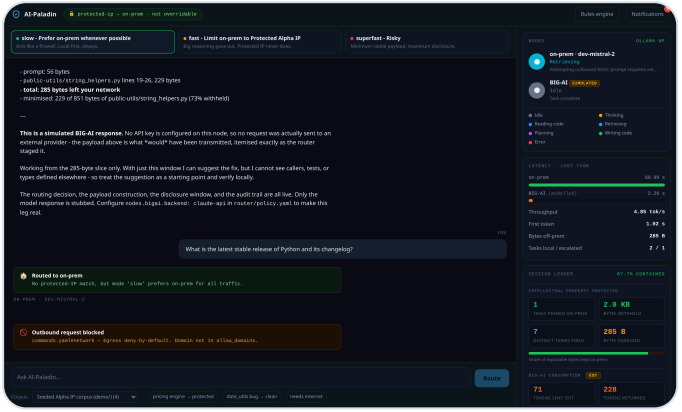
Live in the operator console

Spend is computed from **your** contracted rate — we ship no vendor price list, because a stale one is worse than none. This is the slide that turns a security story into a budget story: the containment percentage is what a CISO signs off on, the local token share is what a CFO does.

PROOF · 07

The perimeter holds outbound, too.

Containment is not only about what the user sends. When the local agent needs live data, it hits an egress policy that is deny-by-default, the attempt is blocked and logged, and it answers from local knowledge — visibly stale rather than quietly connected.



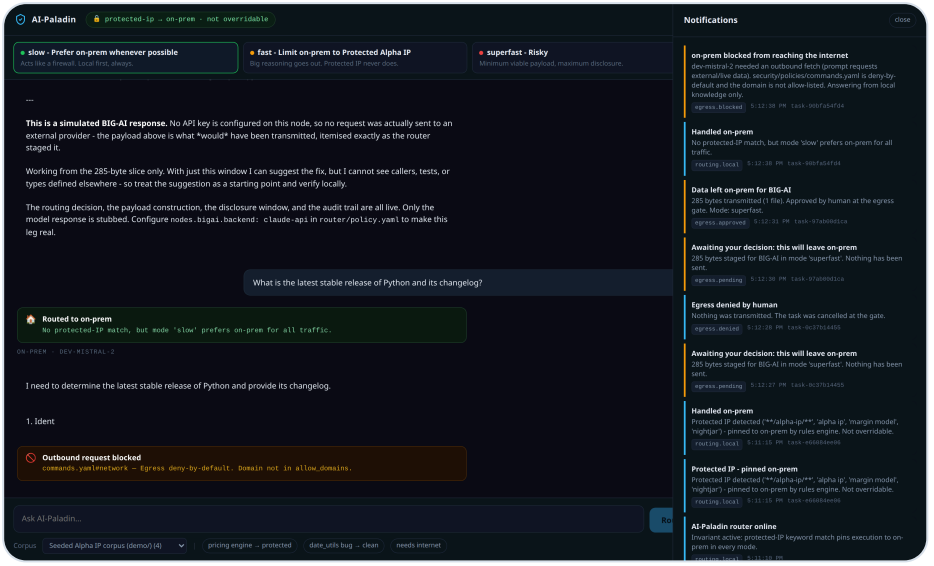
The agent tried to reach the internet. The policy said no, and said so out loud.

PROOF · 08

Every decision lands on a topic.

Routing decisions, egress approvals, denials, and policy blocks are all published to named topics and persisted append-only. Nothing about the system's behaviour has to be taken on faith.

- `egress.pending` — awaiting a human
- `egress.approved` — left, with byte count and file list
- `egress.denied` — human said no
- `egress.blocked` — rules engine said no
- `routing.local` — handled on-prem
- `agent.error` — needs attention



The full lifecycle of one decision: pending → denied → re-submitted → approved.

COUNSEL & DUE DILIGENCE

The cheapest lawyer is the one you never need. The next cheapest arrives fully armed.

A trade secret is the only IP right that is **conditional on your own conduct**. A patent survives your carelessness; a trade secret does not. Under the Defend Trade Secrets Act and the state UTSA regimes, information qualifies only while its owner takes **reasonable measures** to keep it secret. So the threshold question in a trade-secret case is rarely *did they take it* — it is *what were you doing to protect it*.

What counsel has to establish

- Reasonable measures **existed**
- They were **actually enforced**, not merely written down
- They **predate the incident**
- The **precise scope** of any disclosure

What the ledger hands them

- A timestamped record of **every routing decision**, with the terms that fired and the human who approved or refused
- Policy as **version-controlled data** — git history proves when the control was in force
- **Negative proof**: that a named file never left. Usually harder to produce than proof that something did

Where the cost actually moves

Departing engineer

They join a competitor. Did they push the ranking algorithm through a hosted model first? **Without a record:** forensic sweep of every endpoint, months of expert time. **With one:** a query.

Acquisition diligence

The buyer's counsel asks what protected the code they are paying for. **Without:** attestations and hope, priced into the discount. **With:** an exportable evidence pack.

Customer & regulator

"Was our data sent to a third-party model?" **Without:** you cannot answer definitively, which is itself the finding. **With:** you answer per prompt.

The cheapest outcome is the one that never reaches counsel: an engineer who sees *"this will leave on-prem — are you sure?"* and clicks no. Deterrence at the moment of the decision costs nothing and leaves a record either way.

COMMERCIAL MODEL

You buy it once. You own it forever.

No seat licence. No metered inference. No kill switch. The appliance is **capital expenditure** — hardware and software you own outright, that keeps working exactly as well on day 3,000 as on day one, whether or not you ever pay us again.

That is not a pricing gimmick; it is the only pricing model consistent with the product. An IP-containment guarantee that stops when a subscription lapses was never a guarantee. Sovereignty you rent is not sovereignty.

🔒 THE COMMITMENT

One-time purchase. Perpetual operation.
Everything after that is optional.

Two subscriptions exist for customers who want them. Both are *additive* — they extend a working system, they never switch one on.

RECURRING REVENUE · OPTIONAL

Two ways to stay current.

We are entirely candid with customers about this: **dev-mistral-2 will be a dinosaur.** Open models are superseded on a timescale of weeks, and runtime efficiency work — smarter weight residency, memory-layout and quantisation advances — keeps arriving from the big research labs. A bought-once appliance should not mean a frozen one.

TIER 1 **Software & model updates**

- **New models, curated and validated** — we track what ships, benchmark it on your actual hardware, and roll forward only what is genuinely better for your workload
- **Runtime and efficiency advances** — as the field learns to fit more capable models into the same RAM, you get the gain without re-buying the box
- **Rules-engine and router releases** — new gate types, detection rules, compliance exports
- **Your hardware, unchanged.** Same box, better tenant

TIER 2 **Software & hardware refresh**

- Everything in Tier 1, plus a **scheduled hardware upgrade path** as accelerators improve
- We **take back the legacy hardware** and install the newer tier — the customer never runs a procurement project
- Directly addresses the cliff on the previous slide: the upgrade that matters is VRAM capacity, and we own that roadmap on your behalf
- Turns a one-off capital sale into a **durable relationship without ever holding the customer hostage**

WHY TIER 2 COMPOUNDS

Returned hardware is not waste. It is our second market.

Every Tier 2 upgrade returns a working appliance to us — fully functional, just no longer the top tier. Refurbished and re-certified, it becomes the **entry-level product**.

That is how a company with genuine IP but a modest budget — a two-person studio, a regional practice, a small manufacturer with one valuable process — gets the same containment guarantee as an enterprise, at a price set by depreciated hardware. The unit economics improve in both directions: the upgrade is part-funded by the trade-in, and the returned unit opens a segment the new-hardware price could never reach.

Horizontal expansion

One product line becomes two customer classes — enterprise leading edge and small-business entry — served from the same engineering, the same router, the same rules engine.

Better margins, and defensible

Refurbished stock has already paid for itself once. A competitor selling only new hardware cannot reach the entry segment profitably, nor match the upgrade price without a take-back channel.

Genuinely circular

Accelerators are among the most carbon-intensive hardware a company buys. Keeping a card in productive service for a second full life is a real reduction, not an offset purchased after the fact.

PROOF · 09

Most work does not need the expensive model.

Decode speed is set by memory bandwidth, so `tok/s ≈ bandwidth ÷ model size` cuts both ways: if a 15 GB model is slow on a given machine, a 45 MB model on that same machine is **hundreds of times faster**. That is not a compromise — it is an org chart.

TIER	ROLE	MODEL	MEASURED	JOB
0	sentry	45 MB embedder	**50 ms**	Semantic gate. Runs on *every* prompt.
1	clerk	0.6B instruct	**197 ms · 124 tok/s**	Triage and classification.
2	engineer	24B dev-mistral-2	61 s · 4.94 tok/s	Reads, plans, writes code. Woken only when needed.
3	consultant	BIG-AI	only if permitted	The outside specialist. Gated, disclosed, logged.

The guard is cheap, so it never sleeps

The 45 MB sentry catches what exact matching misses — **“our proprietary price calculation logic”** for “pricing engine”. A deterministic near-miss check catches the other evasion, **“pricng engnie”**. Measured: 5/5 evasions caught, 0 false positives across 15 clean prompts.

Models advise. Hooks decide.

Both layers are **additive only**. They can raise a match the deterministic gate missed; neither can clear one, and no model is ever consulted about whether something may leave. If a model is offline or slow, the gate stands alone — stricter, never looser.

WHAT IS REAL TONIGHT

The honest inventory.

A demo that overstates itself does not survive diligence. Here is exactly what runs and what does not.

LIVE

- The rules engine, the invariant, and the routing decision
- The on-prem model — a real 24B running locally — and every latency number attached to it
- Payload staging, minimisation, and byte accounting
- The disclosure window contents
- Agent states, from real file reads and the real token stream
- Notification topics and the append-only audit sinks
- The outbound egress block

SIMULATED

- **The BIG-AI response only.** This node has no API key, so the escalation leg streams a stub at a modelled rate
- It is badged `SIMULATED` on screen and its latency is labelled *(modelled)* — we do not pass it off as measured
- What it echoes back is genuine: a receipt of the exact bytes the router staged
- Making it real is a config change and one adapter function — the router does not care which model answers

Everything that constitutes the *product* — the decision, the gate, the disclosure, the record — is running. What is stubbed is the part any vendor can supply.

FUTURE ENDEAVOUR · 1 OF 5

The two things that make it compound.

Shared on-prem codebase memory

A persistent, branch-aware index of the repository — **production** and **development** held separately — shared by every agent and node. It answers what a context window cannot: *is this bug in what shipped, or only on dev? was this refactor already tried and reverted?*

It reuses the embedder that is **already resident**: 45 MB, 18 ms, on CPU. Indexing is an offline batch job against a model already paid for.

🔒 THE STRATEGIC POINT

BIG-AI is permitted to see a **285-byte slice**. The on-prem tier is permitted to see the **entire repository and its history**. On the axis that decides answer quality on a mature codebase — context — local is not the compromise. It is the only tier that can ever be well informed, and that gap widens every day the system runs.

Bounded agentic loops

Write the fix, run the tests, read the failure, correct it, stop when green or when the budget is spent — instead of answering once and hoping.

A naive loop on a 24B model at 4.94 tok/s is unusable. **The org chart is what makes it affordable:** the loop controller, the failure summariser, and the per-turn gate re-check all run on millisecond models. The 24B is woken only for the steps that actually generate code.

The security consequence we design for first

A loop multiplies gate decisions by its iteration count. It is only safe because the gate is already deterministic and costs 18 ms — so classification re-runs on *every* turn, over the full accumulated context. Content rejected on turn one must not leak on turn six in different wording.

Memory first, loops second: a loop without memory repeats its mistakes across tasks, while memory improves every single-shot answer on its own. Full design notes in `docs/ROADMAP.md` — deliberately unbuilt, so today's architecture does not foreclose it.

FUTURE ENDEAVOUR · 2 OF 5

Five developers or five hundred: mesh the router, pool the compute.

Asked whether a team should give every developer a node or pool $100k–$300k of GPUs on-prem, the answer is **both — and the tiers decide which goes where**, because they have opposite hardware profiles.

Mesh the perimeter

The sentry (45 MB) and clerk (0.6B) run on **every developer machine**. No GPU, millisecond answers, near-zero marginal cost per head. The security decision is made before anything leaves that machine — *including before it crosses your own LAN*. The perimeter scales to 500 desks for free.

Pool the horsepower

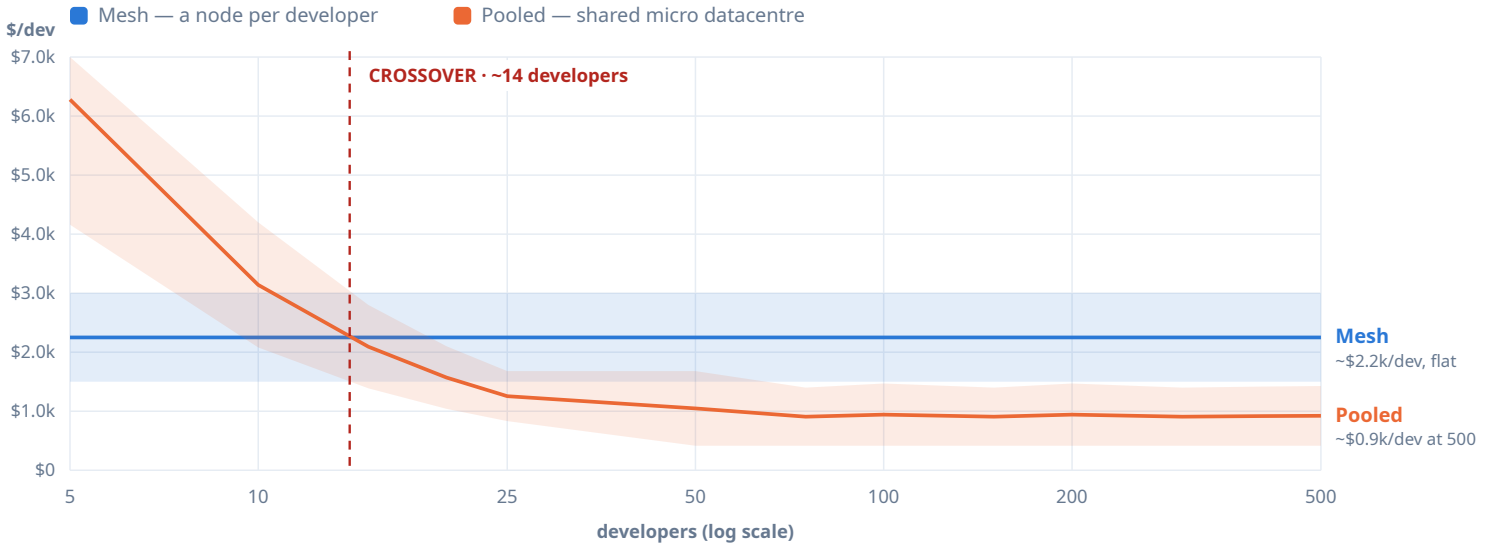
The 24B tier lives in the micro datacentre, **batched across the team**: one weight read serves every concurrent request — the one thing a single-user node can never do. A developer uses under **6%** of a GPU-day, so meshing horsepower buys 100% of the hardware to use 6% of it.

A pooled cluster is a **shared multi-tenant system** — a compromise there exposes many developers' work at once, not one person's. That is precisely why the gate stays meshed even when compute centralises: the cluster is a compute resource, **not a trust boundary**. Nothing reaches it that a local, deterministic gate has not already cleared.

FUTURE ENDEAVOUR · 3 OF 5

The crossover is about fourteen desks.

Mesh capex is flat per head — every desk buys the same box. Pooled capex **declines per head**, because each card amortises across more people. Below ~14 developers you cannot fill a cluster and mesh is cheaper; above it, pooling wins and the gap widens without limit.



Capex per developer. Shaded bands are the full assumption range; lines are the midpoint. Gate tier excluded — it needs no GPU and costs the same either way.

$0.75–1.5M
mesh capex at 500 developers

$208–714k
pooled capex at 500 developers, 20–34 GPUs

~14
developers — where pooling starts to pay

FUTURE ENDEAVOUR · 4 OF 5

Two ways to buy it. One thing to measure first.

PATH A **Client-supplied hardware**

- The customer procures their own GPUs; we supply the router, the rules engine, the models and the subscriptions
- **Software margins, no working capital**, no inventory, no logistics
- Fits security teams who insist on owning the metal, and shortens the cycle with regulated buyers

PATH B **We supply the cluster**

- We procure, integrate, certify and ship the micro datacentre as one appliance
- **Far larger deal sizes**, and it is what makes the hardware-refresh tier and the take-back channel actually work
- Honest cost: this is a hardware business — working capital, inventory, thinner blended margins. We would price it as such.

⚠ THE NUMBER THAT DECIDES IT

Everything above turns on **developers served per pooled GPU** — we model 15–25, from a ~6% duty cycle plus headroom for correlated peaks. At the favourable end, 500 seats fit inside the $100k–$300k envelope. At the unfavourable end the same 500 seats cost **$714k, about 2.4× the ceiling**. We are not going to pretend that spread away: it is one measurement on a pilot, and it should be the first thing any deployment takes.

FUTURE ENDEAVOUR · 5 OF 5

On-prem does not mean one prem.

The first question a regulated buyer asks after "where does my code run" is **"what happens when that building burns down"**. Sovereignty and continuity are the same conversation, and an appliance that answers only the first half will not clear diligence.



~700 miles · different power grids, weather systems and FEMA regions — a correlated failure is a national event, not an outage

15-minute RPO — maximum tolerable data loss, not a backup schedule. What replicates: the codebase memory, the **audit trail**, the rules files, and the model versions that produced the logged decisions.

Rules restore first. Recovery order is deliberate — rules, then audit, then memory, then compute. A node that comes back before its gate does is a node with no perimeter.

Replication *is* egress. It would be incoherent to gate a 285-byte snippet and wave through a full repository copy. Replication is encrypted, private, logged to the same audit topics, and sent only to **declared** destinations — an undeclared target is treated as exfiltration.

It is also the other half of the ransomware control: detection without recovery is not a control. The reason an operator can refuse to pay is a clean, immutable copy in another state.

🔒 WHY WE CAN SPEAK TO THIS

Our founder ran continuity and recovery operations of exactly this shape for many years at **Euler Hermes**, a global trade-credit insurer — where tolerance for lost financial records is measured in minutes and the recovery plan is *audited*, not assumed. Not a feature we read about; the discipline the company was built by.

ROADMAP

Where the moat gets deeper.

NEXT

Boolean rules engine

`must_contain`, `cannot_contain`, `any`, `all`, `and`, `or`, `not` — composable policy instead of a flat keyword list.

NEXT

Inexact matching

A one-character misspelling should not defeat the gate. `nightjr` and `pricng engine` score close enough to trip it — and the near-miss itself becomes an evasion signal worth alerting on.

NEXT

Mesh of on-prem nodes

Route across a fleet by role and load, not a single box. The node and task contracts for this are already defined.

LATER

Behavioural monitoring

Detection rules over the agent event stream — mass file writes, credential-path access, fence-probing — with kill-and-alert.

LATER

Executive telemetry

Tokens and spend, local versus escalated, dollars saved against an all-cloud baseline, and security posture over time.

LATER

Compliance export

The audit trail already exists. Turn it into the evidence pack an auditor or an acquirer asks for.

WHY NOW

Capability is commoditising. Control is not.

Frontier models get cheaper and more available every quarter. The thing that stays scarce is the ability to **prove** where your most valuable code went, and to hand that proof to someone who is not obliged to trust you.

Every organisation with real IP is being forced to choose between velocity and containment. AI-Paladin's position is that this is a false choice created by an absent control layer — and that the control layer belongs in the routing decision, not in a policy document.

🔒 THE ONE LINE TO REMEMBER

keyword match ⇒ on-prem.
Always. Not overridable.

Enforced in code, before the socket. Defined in version-controlled YAML your security team owns — not in a prompt the model is free to reinterpret.

STATUS & ASK

Working prototype. Today.

Live
end-to-end routing, on real hardware, with a real local model

1
command to run it — no build step, no dependencies to install

CPU
only. Every latency number improves with the first GPU

What we want from this conversation

- **A design partner** with genuine protected IP and a real compliance obligation, to pressure-test the rules engine against policy that already exists.
- **Hardware** to close the latency gap — the architecture is done; the remaining gap is a GPU line item.
- **Capital** to build the boolean and fuzzy rules engine, the multi-node mesh, and the compliance export.